VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Shivanya Tomar	24BCE10026
Slot	**Date of Submission**
B22	31/03/2026

1. Introduction

Open-source software has become a quiet but powerful foundation of the digital world. From operating systems and browsers to databases and developer tools, many of the systems people use every day are built and maintained in the open by global communities rather than by one closed organization. The open model matters because it allows users to inspect the code, improve it, share it, and learn from it. This creates a culture of collaboration that often leads to faster innovation and stronger trust.

For this project, I selected **Git** as the software to study. Git is one of the most influential open-source tools in modern computing because it helps developers track changes, manage source code history, and work together without losing control of project versions. It is used by students, startups, enterprises, and large open-source communities alike, which makes it a strong example for understanding both the technical and philosophical value of open software.

This report explores Git from several angles. It looks at why Git was created, what technical problem it solved, how its license protects openness, and how it fits into the larger free and open-source ecosystem. Along with the conceptual analysis, the project also includes five original shell scripts that demonstrate practical Linux and Bash scripting skills in a clear and application-oriented way.

The goal of this capstone is not only to describe Git as a tool, but also to understand what it represents. Git shows how open development can solve real-world problems at scale, and how shared knowledge can produce software that becomes essential across the world

2. Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

Part A — Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Git was created in 2005 by Linus Torvalds during the development of the Linux kernel. Before Git existed, the Linux kernel community used a proprietary version control system called BitKeeper. That arrangement broke down due to licensing disagreements, and the community suddenly needed a replacement that was fast, reliable, and suitable for large-scale distributed development.

This was not a small requirement. Kernel development involved many contributors, frequent changes, and a strong need for data integrity. Existing systems were either too centralized, too slow, or too limited for the scale of Linux development. Torvalds responded by designing Git with a clear focus on speed, distributed workflows, and strong change tracking.

One of the defining ideas behind Git is that every developer can have a complete local copy of the repository, including its history. This means users can work offline, inspect previous states, and create branches without depending constantly on a central server. That design made Git practical, efficient, and resilient.

Git was also released as open-source software, which was a deeply meaningful choice. Since the problem itself emerged from dependence on a proprietary tool, the solution was intentionally built in a way that the community could own, maintain, and improve together. That decision helped Git grow from a replacement utility into a global standard for version control.

Another remarkable detail is how quickly Git was initially built. Even in its early form, the tool reflected strong engineering priorities: content integrity, speed of operation, and support for branching and merging. These strengths continue to define Git today.

A2. The license — what it actually says

Git is distributed under the **GNU General Public License version 2 (GPL v2)**. This is a copyleft license, which means users are free to use, study, modify, and share the software, but any distributed modified version must also remain under the same license terms.

The license protects four key freedoms commonly associated with free software: the freedom to run the program, the freedom to study how it works, the freedom to modify it, and the freedom to redistribute it. These freedoms matter because they prevent the software from becoming locked away from the very community that builds value around it.

In Git's case, GPL v2 is especially appropriate. Git is infrastructure software used by developers around the world. If a company could take Git, improve it privately, and release a closed version without sharing changes, the broader ecosystem could lose access to valuable improvements. The GPL reduces that risk by keeping distributed improvements open.

This is one of the major differences between GPL and permissive licenses such as MIT or BSD. Permissive licenses allow broader reuse in proprietary software, while GPL prioritizes preserving openness downstream. For a tool as foundational as Git, this stronger protection helps preserve community ownership over time.

In my view, the GPL fits Git well because Git is not just a product; it is a public infrastructure tool for collaboration. A license that protects openness supports the same cooperative spirit that made Git successful in the first place.

A3. The ethics of open source

Open-source software is often described only in technical terms, but it also raises ethical questions. At its best, open source reflects values such as transparency, shared learning, accountability, and collective improvement. Git supports those values by making collaborative development practical and dependable across different people and organizations.

In cybersecurity, openness has both strengths and risks. On one hand, publicly visible code can be inspected by researchers, maintainers, and contributors who identify problems and improve security. On the other hand, attackers can also examine the same code. This means security in open-source systems depends less on secrecy and more on active review, timely patching, and responsible maintenance.

This creates an important ethical balance. Not every system can expose all details without concern, especially when sensitive infrastructure or privacy-related components are involved. Still, the general principle behind open source remains powerful: systems become healthier when more knowledgeable people can study and improve them.

Open-source business models also show that ethics and sustainability do not have to conflict. Companies such as Red Hat and others have shown that it is possible to build services, support models, and enterprise offerings around open ecosystems while still contributing back. That reinforces the idea that openness can be practical, not just idealistic.

The phrase “standing on the shoulders of giants” applies strongly here. Git itself is a tool that helps communities reuse, refine, and extend each other’s work. In that sense, Git does not only manage code. It enables a culture where progress is shared instead of repeatedly rebuilt from zero.

Part B — Linux Footprint (Unit 2)

Git is generally installed on Linux through a package manager such as apt, dnf, or pacman, depending on the distribution. In most systems, installation is straightforward because Git is packaged as a standard command-line utility and does not require a background daemon for ordinary use.

The primary executable is commonly located at:

```
/usr/bin/git
```

Git works as a user-invoked tool rather than a continuously running service. When a command such as git status, git clone, or git commit is executed, the program runs in the current shell session and acts on the repository located in the working directory.

A repository managed by Git contains a hidden .git directory. This directory stores commit history, references, configuration details, objects, logs, and metadata required for version control operations. That hidden directory is effectively the internal database of the repository.

Git also interacts with several system-level features. It relies on standard file permissions, uses SSH or HTTPS for remote communication, and integrates naturally with editors, IDEs, CI/CD systems, and shell environments. Updates are usually received through the operating system package manager or through official installation packages.

From a system perspective, Git is lightweight compared to large server software, but its impact is much larger than its footprint. It inserts itself into the daily workflow of developers and becomes a control layer for collaboration, history tracking, and release discipline.

Part C — The FOSS Ecosystem (Units 3 & 4)

Git occupies a central place in the free and open-source software ecosystem. While Git itself is a version control system, its importance grows because so many other platforms and workflows are built around it. Hosting services such as GitHub and GitLab, collaborative review systems, deployment pipelines, and open-source project communities all depend on Git-based workflows.

Git is also a good example of how a relatively focused tool can shape a much larger environment. It does not try to be an all-in-one development platform. Instead, it performs version control extremely well and becomes the base layer on top of which many other systems operate.

The ecosystem around Git includes:

- GitHub for hosting, pull requests, issue tracking, and project visibility.
- GitLab for integrated development and DevOps workflows.
- Bitbucket for repository management in team-based settings.
- CI/CD pipelines that trigger testing and deployment from repository events.
- IDE integrations that make version control part of everyday coding.

In a LAMP-style development environment or any comparable stack, Git helps manage source code changes cleanly across local development, staging, and production environments. Developers use it to coordinate teamwork, track regressions, maintain release branches, and preserve accountability for changes over time.

One of Git’s strongest ecosystem advantages is its community. It is supported by extensive documentation, books, tutorials, forum discussions, and years of practical use by millions of developers. That depth of community support makes Git not only useful, but sustainable.

Part D — Open Source vs Proprietary (Critical Analysis)

Feature	Git (Open Source)	BitKeeper (Proprietary)
Cost	Freely available	Commercial licensing required
SourceCode Access	Open for inspection	Closed to the public
Modification Freedom	Users can adapt and improve it	Users cannot freely modify the core software
Development Model	Community-supported and transparent	Vendor-controlled
Collaboration Style	Distributed and flexible	More restricted by product design and licensing
Security Review	Can be examined publicly	Largely dependent on the vendor
Long-Term Trust	Shared across the ecosystem	Tied closely to one company

Git offers a more durable and flexible model for collaborative software development because it combines technical performance with openness. Its source code can be studied, its behavior can be audited, and its development is not locked to a single commercial actor.

A proprietary system may offer polished tooling or dedicated support, but it also creates dependency. If pricing changes, licenses become restrictive, or development priorities shift, users have limited control. Git avoids that weakness by belonging to the wider software community rather than to one owner.

From both a practical and philosophical perspective, Git is a stronger long-term choice. It supports individual learning, professional collaboration, and large-scale software engineering while preserving user freedom. For these reasons, I would confidently recommend Git for real-world use and for participation in open-source development.

3. Shell Script Tasks

This part of the project contains five original shell scripts written to demonstrate practical scripting concepts in a Linux environment. The scripts are intentionally simple to understand, but each one addresses a realistic task related to system inspection, software usage, automation, or user interaction.

The aim is not to create overly complex code. Instead, the aim is to show correct use of variables, loops, conditions, input handling, text processing, and file operations in a clear and meaningful way.

Script 1 — System Identity Report (Unit 1 & 2)

Purpose:
This script creates a quick environment summary for a student working with Git on Linux. It displays personal details, Git version, current shell, machine name, and present working directory.
Concepts Used:

- Variables
- Command substitution
- if condition
- Output formatting
- Basic command execution

Starter structure

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Shivanya Tomar | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Shivanya Tomar"
ROLL_NO="24BCE10026"
SOFTWARE_CHOICE="Git"

# --- System Info ---
KERNEL=$(uname -r)                              # Get kernel version
USER_NAME=$(whoami)                             # Get current logged-in user
HOME_DIR=$HOME                                  # Get home directory path
UPTIME=$(uptime -p 2>/dev/null || uptime)       # Get uptime, fallback if -p not
supported
DATE=$(date)                                    # Get current date and time

# Get OS distribution name from /etc/os-release
if [ -f /etc/os-release ]; then
    # Extract PRETTY_NAME value, remove quotes
    DISTRO=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d= -f2 | tr -d '"')
else
    DISTRO="Unknown Linux Distribution"
fi

# --- Display ---
echo "==========================================="
echo "     System Identity Report"
echo "==========================================="
echo "Student Name     : $STUDENT_NAME"
echo "Roll Number      : $ROLL_NO"
echo "Software         : $SOFTWARE_CHOICE"
echo "OS Distribution  : $DISTRO"
echo "Kernel Version   : $KERNEL"
echo "User             : $USER_NAME"
echo "Home Directory   : $HOME_DIR"
echo "Uptime           : $UPTIME"
echo "Date & Time      : $DATE"
echo "License          : GNU General Public License v2 (GPL v2)"
echo "==========================================="
```

Output:

```
pradeepti_s@Sphynxia:/mnt/c/Users/Pradeepti S/oss-audit$ ./script1.sh
========================================
     System Identity Report
========================================
Student Name     : Shivanya Tomar
Roll Number      : 24BCE10026
Software         : Git
OS Distribution  : Ubuntu 24.04.4 LTS
Kernel Version   : 6.6.87.2-microsoft-standard-WSL2
User             : pradeepti_s
Home Directory   : /home/pradeepti_s
Uptime           : up 4 minutes
Date & Time      : Tue Mar 31 16:09:43 UTC 2026
License          : GNU General Public License v2 (GPL v2)
========================================
```

Working:

This script generates a comprehensive welcome screen that displays essential system information. It begins by defining variables for student details and then uses command substitution to capture system information. The kernel version is obtained using the uname -r command, while whoami retrieves the current logged-in user. The script accesses the $HOME environment variable to get the user's home directory and uses uptime -p for a human-readable uptime display. The current date and time are captured with the date command.

For determining the Linux distribution, the script checks if the /etc/os-release file exists—a standard file on modern Linux systems. It then uses grep to extract the PRETTY_NAME field, followed by cut and tr commands to isolate the value and remove quotation marks. If the file doesn't exist, it defaults to "Unknown Linux Distribution".

The final section formats and displays all collected information with clear headings and separators using echo statements. This script demonstrates the use of variables for storing data, command substitution $() to execute commands and capture output, basic output formatting, and conditional file checking. It successfully creates a professional system identity report that runs without errors on any Linux system.

Script 2 — FOSS Package Inspector (Unit 2)

Purpose:
This script checks whether the current directory is a Git repository. If it is, the script shows the branch name and short repository status.

Concepts Used:

- if-else condition
- Command substitution
- Git commands
- Output formatting
- Exit status checking

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Shivanya Tomar

PACKAGE="git"   # chosen software

echo "Checking package: $PACKAGE"
echo "----------------------------"

# Check if package is installed (works on Fedora/RHEL with rpm)
# For Debian/Ubuntu, replace with: dpkg -l $PACKAGE &>/dev/null
if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."

    echo "Package Details:"
    # Extract name, version, summary, and license from package info
    rpm -qi $PACKAGE | grep -E 'Name|Version|Summary|License'

else
    echo "$PACKAGE is NOT installed."
fi

echo ""
echo "Philosophy Note:"
# Case statement providing a short description based on package name
case $PACKAGE in
    git) echo "Git: a distributed version control system that enables collaboration
and code sharing." ;;
    httpd) echo "Apache: the web server that built the open internet." ;;
    mysql) echo "MySQL: open-source database used in millions of applications." ;;
    firefox) echo "Firefox: a browser that supports an open and free web." ;;
    *) echo "Unknown software." ;;
esac
```

Output:

```
pradeepti_s@Sphynxia:/mnt/c/Users/Pradeepti S/oss-audit$ ./script2.sh
Checking package: git
----------------------------
git is installed.
Package Details:
Package: git
Version: 1:2.43.0-1ubuntu7.3
Description: fast, scalable, distributed revision control system

Philosophy Note:
Git: a distributed version control system that enables collaboration and cod
e sharing.
```

Working:

This script checks whether a specified open-source software package (Git in this case) is installed on the system and provides detailed information about it. The script defines a variable PACKAGE set to "git" as the software to inspect. It then uses an if statement with rpm -q to check if the package is installed.

The &>/dev/null redirects suppresses both standard output and error messages, allowing the script to only evaluate the exit status—0 if installed, non-zero if not.
If the package is installed, the script executes rpm -qi to query detailed package information. The output is piped to grep -E which filters for lines containing "Name", "Version", "Summary", or "License", presenting only the most relevant package details. If the package is not installed, it simply reports that fact.

Following the package check, the script uses a case statement to display a philosophical

description based on the package name. The case structure evaluates the value of $PACKAGE and matches it against predefined patterns (git, httpd, mysql, firefox), executing the corresponding echo statement. A wildcard * pattern catches any unspecified package names. This script demonstrates conditional execution with if-then-else, pattern matching with case statements, command piping, and the use of package management commands in shell scripting.

Script 3 — Disk and Permission Auditor (Unit 2)

Purpose:
This script displays a compact view of recent Git commits from a repository, helping the user inspect project activity quickly.
Concepts Used:

- Variables
- Positional arguments
- Default values
- if condition
- Git log formatting

Starter structure

```
        #!/bin/bash
        # Script 3: Disk and Permission Auditor
        # Author: Shivanya Tomar

        # List of important system directories to audit
        DIRS=("/etc" "/var/log" "/home" "/usr/bin"
    "/tmp")

        echo "Directory Audit Report"
        echo "----------------------"

        # Loop through each directory in the list
        for DIR in "${DIRS[@]}"; do
            if [ -d "$DIR" ]; then
                # Get permissions, owner, and group using
    ls -ld
```

```
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3,
$4}')
        # Get disk usage in human-readable format
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size:
$SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo ""
echo "Git Binary Check:"
# Check location and permissions of git binary
if [ -f "/usr/bin/git" ]; then
    ls -l /usr/bin/git
else
    echo "Git binary not found"
fi
```

Output:

```
pradeepti_s@Sphynxia:/mnt/c/Users/Pradeepti S/oss-audit$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 34M
/home => Permissions: drwxr-xr-x root root | Size: 416K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 20K

Git Binary Check:
-rwxr-xr-x 1 root root 4066232 Jul  2  2025 /usr/bin/git
```

Working:

This script performs an audit of important system directories by iterating through a predefined list and reporting both disk usage and permission details. The script begins by defining an array DIRS containing five critical directories: /etc, /var/log, /home, /usr/bin, and /tmp. It then uses a for loop to iterate through each directory in this array.

For each directory, the script first checks if it exists using the -d test operator within an if statement. If the directory exists, it executes two commands: ls -ld to retrieve detailed directory information including permissions, owner, and group; and du -sh to calculate the directory's disk usage in a human-readable format. The output from ls -ld is piped to awk to extract the permission string (column 1), owner (column 3), and group (column 4). The disk usage output is piped to cut -f1 to extract just the size value, discarding the directory name that du typically appends. Both values are stored in variables and displayed in a formatted line.

If a directory does not exist on the system, the script reports this instead. After completing the directory loop, the script performs a separate check for the Git binary at /usr/bin/git, displaying its detailed listing using ls -l if found. This script demonstrates the use of arrays, for loops, conditional directory checks, command substitution, and text processing with awk and cut.

Script 4 — Log File Analyzer (Units 2 & 5)

Purpose:
This script scans the current directory and counts how many files belong to common source-

code or text categories such as .sh, .py, .java, and .txt.

Concepts Used:

- for loop
- case statement
- Arithmetic operations
- File testing
- Output formatting

Starter structure

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Shivanya Tomar
# Usage: ./script4.sh logfile [keyword]

LOGFILE=$1                      # First argument: log file path
KEYWORD=${2:-"error"}           # Second argument: keyword, default to "error"
COUNT=0

# Check if log file is provided
if [ -z "$LOGFILE" ]; then
    echo "Error: No log file specified."
    echo "Usage: $0 <logfile> [keyword]"
    exit 1
fi

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "File is empty."
    exit 1
fi
```

```
# Read file line by line and count matches
while IFS= read -r LINE; do
    # Case-insensitive match for keyword
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo ""
echo "Last 5 matching lines:"
# Show last 5 occurrences with case-insensitive grep
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

Output:

```
pradeepti_s@Sphynxia:/mnt/c/Users/Pradeepti S/oss-audit$ ./script4.sh /var/l
og/syslog
Keyword 'error' found 12 times in /var/log/syslog

Last 5 matching lines:
2026-03-30T21:38:06.128371+00:00 Sphynxia kernel: WSL (244) ERROR: CheckConn
ection: getaddrinfo() failed: -5
2026-03-31T16:05:28.529495+00:00 Sphynxia systemd[1]: apport-autoreport.path
 - Process error reports when automatic reporting is enabled (file watch) wa
s skipped because of an unmet condition check (ConditionPathExists=/var/lib/
apport/autoreport).
2026-03-31T16:05:28.529499+00:00 Sphynxia systemd[1]: apport-autoreport.time
r - Process error reports when automatic reporting is enabled (timer based)
was skipped because of an unmet condition check (ConditionPathExists=/var/li
b/apport/autoreport).
2026-03-31T16:05:28.532027+00:00 Sphynxia kernel: RAS: Correctable Errors co
llector initialized.
2026-03-31T16:06:14.305369+00:00 Sphynxia kernel: WSL (234) ERROR: CheckConn
ection: getaddrinfo() failed: -5
```

Working:

This script analyzes a log file to count how many lines contain a specific keyword. The log file name is provided as a command-line argument, and an optional keyword can also be specified. If no keyword is given, the script uses "error" as the default.

The script first checks whether the file exists using an if condition. It also verifies whether the file is empty before processing it. If the file is missing or empty, the script exits with an appropriate message.

The main logic uses a while read loop to process the file line by line. For each line, it checks whether the keyword is present using the grep command. If a match is found, a counter variable is incremented.

After processing the file, the script prints the total number of matching lines. It also displays the last five lines containing the keyword using a combination of grep and tail.

This script demonstrates how to handle file input, use loops, apply conditional checks, and work with command-line arguments in shell scripting. It successfully processes log files and provides useful summary statistics about error or warning occurrences.

Script 5 — The Open Source Manifesto Generator (Unit 5)

Purpose:
This script asks the user a few questions and generates a short personal reflection about open-source software in a text file.

Concepts Used:

- User input with read
- Variables
- Command substitution
- File output redirection
- Displaying saved content

Starter structure

```
#!/bin/bash
# Script 5: Open Source Manifesto
Generator
# Author: Animesh Daiman

echo "Answer three questions to
generate your manifesto."
echo ""

# Take user input
read -p "1. Name one open-source tool
you use every day: " TOOL
read -p "2. In one word, what does
'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would
build and share freely: " BUILD

# Get current date
DATE=$(date '+%d %B %Y')

# Output file name based on current
user
OUTPUT="manifesto_$(whoami).txt"

# Compose and save the manifesto
{
    echo "On $DATE, I reflect on the
importance of open source."
    echo "I regularly use $TOOL, which
helps me in my daily work."
    echo "For me, freedom means
$FREEDOM."
    echo "I believe in building $BUILD
and sharing it openly so that others
can learn, improve, and collaborate."
    echo "Open source is not just
about code, but about community and
knowledge sharing."
} > "$OUTPUT"

# Alias concept: This script could use
aliases like 'll' for listing, but
that's a shell feature.
# alias ll='ls -l'

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat "$OUTPUT"
```

Output:

```
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Red Pill
3. Name one thing you would build and share freely: Tension.txt

Manifesto saved to manifesto_pradeepti_s.txt

On 31 March 2026, I reflect on the importance of open source.
I regularly use Git, which helps me in my daily work.
For me, freedom means Red Pill.
I believe in building Tension.txt and sharing it openly so that others can l
earn, improve, and collaborate.
Open source is not just about code, but about community and knowledge sharin
g.
```

Working:

This interactive script generates a personalized open-source philosophy statement by asking the user three questions and composing a manifesto based on their responses. The script begins by displaying a welcome message and then uses the read command with the -p option to prompt the user for three pieces of information: an open-source tool they use daily, their personal definition of freedom (in one word), and something they would build and share freely.

After capturing the user's input into variables (TOOL, FREEDOM, BUILD), the script uses the date command with custom formatting to generate the current date in a readable format (e.g., "31 March 2026"). It then constructs an output filename using the current username, resulting in a unique manifesto file per user.

The manifesto composition uses a command block enclosed in curly braces {} with output redirection > to write all lines to the specified file. This approach is more efficient than multiple echo statements with individual redirects. The script composes a meaningful paragraph that incorporates all three user responses, reflecting on the importance of open source software, community collaboration, and knowledge sharing.

Finally, the script confirms the file creation, displays the saved filename, and outputs the complete manifesto content using the cat command. A commented alias concept demonstrates understanding of shell features like aliases. This script effectively demonstrates interactive user input with read, string concatenation, file output redirection, variable usage, and the creation of personalized dynamic content in shell scripting.

4. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Choice confirmed*
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft A2 + A3*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Scripts 1–2 working*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Scripts 3–4 working*
7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Complete draft*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission*

5. Reference Resources

The following resources were used to understand Git, open-source software principles, licensing models, and Bash scripting concepts relevant to this project. The formatting style follows the reference-oriented structure used in the sample document.

Git and Version Control

- Git Official Website.
  Available at: https://git-scm.com
  Used to understand Git's purpose, installation guidance, and core features.
- Git Documentation.
  Available at: https://git-scm.com/docs
  Used to study Git commands and command behavior in practical workflows.
- Chacon, Scott and Straub, Ben. *Pro Git Book*.
  Available at: https://git-scm.com/book/en/v2
  Used to understand Git architecture, branching, commits, and repository behavior.
- GitHub Documentation.
  Available at: https://docs.github.com
  Used to connect Git concepts with real collaborative development practices.

Open Source Philosophy and Licensing

- GNU Project. *The Free Software Definition*.
  Available at: https://www.gnu.org/philosophy/free-sw.html
  Used to understand the idea of software freedom and user rights.
- Open Source Initiative. *The Open Source Definition*.
  Available at: https://opensource.org/osd
  Used to study the principles that define open-source licensing.
- Choose a License.
  Available at: https://choosealicense.com
  Used for understanding differences between copyleft and permissive licenses.
- SPDX License List.
  Available at: https://spdx.org/licenses/
  Used to confirm license naming and standard references.

Linux and Shell Scripting

- GNU Bash Manual.
  Available at: https://www.gnu.org/software/bash/manual/
  Used to review shell syntax, variables, loops, and conditions.
- The Linux Command Line by William Shotts.
  Available at: https://linuxcommand.org
  Used to strengthen understanding of Bash commands and terminal workflows.
- Linux manual pages (man pages).
  Accessed through commands such as man bash, man grep, man find, and man git.
  Used to understand real command usage and option details.

Additional Learning Resources

- Fedora Project Documentation.
  Available at: https://docs.fedoraproject.org
  Used for Linux package management and environment understanding.
- Online Bash scripting examples and command-line practice resources.
  Used to refine script structure, improve logic, and test alternative approaches